

Security Fixes Applied

Week 2 — Side-by-side walk-through
with live verification proof

Author: Maryam Mueen
Intern ID: DHC-1151

1 Quick Reference

Every Week 1 finding is mapped to the fix in `secure-app/`. All fixes were verified with live `curl` runs and the captured response strings are reproduced inline.

Finding	Fix in <code>secure-app/</code>	Library
V-01 SQLi	Parameterised <code>db.prepare(...).run(?, ?, ?)</code>	better-sqlite3
V-02 XSS	EJS switched to <code><%= %></code> , strict CSP added	helmet, EJS
V-03 Plaintext PW	<code>bcrypt.hash(password, 12) + compare</code>	bcrypt
V-04 Missing headers	<code>helmet(contentSecurityPolicy: ...)</code>	helmet
V-05 Framework leak	<code>app.disable('x-powered-by')</code>	built-in
V-06 Insecure cookie	JWT + <code>HttpOnly</code> + <code>SameSite=Strict</code> + <code>Secure</code>	jsonwebtoken
V-07 Public <code>/users</code>	<code>requireLogin</code> + username column only	in-house
V-08 No validation	<code>validator.isEmail / isAlphanumeric / isStrongPassword</code>	validator
V-09 No rate limit	<code>express-rate-limit</code> on <code>/login</code> + <code>/signup</code>	express-rate-limit
Week 3 logging	winston JSON-line logger + colour console	winston

2 Fix-by-Fix Walk-through

2.1 Fix V-01 — Parameterised SQL queries

Before (vulnerable)

```
const sql = "SELECT * FROM users WHERE username = '" + username +
            "' AND password = '" + password + "'";
const user = db.prepare(sql).get();
```

After (secure)

```
const user = db.prepare('SELECT * FROM users WHERE username = ?')
    .get(username);
```

Proof

```
$ curl -i -X POST http://localhost:3000/login \
  --data-urlencode "username=admin' OR '1'='1" \
  --data-urlencode "password=anything"
HTTP/1.1 200 OK
...
<p class="err">Invalid credentials</p>
```

The injection attempt no longer authenticates — the input is bound as a parameter, not parsed as SQL.

2.2 Fix V-02 — Output escaping and strict CSP

EJS variables were changed from the unescaped `<- user.bio %>` to the escaped `<%= user.bio %>` tag, and helmet was configured with a Content-Security-Policy that blocks inline scripts:

```
helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self'],
      objectSrc: ['none'],
```

```

    ...
  },
},
})

```

Proof

Posting bio `<script>alert('XSS')</script>` and re-fetching the profile page yields:

```
<div class="bio-box">&lt;script&gt;alert(&#39;XSS&#39;)&lt;/script&gt;</div>
```

The payload renders as literal text. It is never executed.

2.3 Fix V-03 — bcrypt password hashing

```

const BCRYPT_COST = 12;
const hash = await bcrypt.hash(password, BCRYPT_COST);
db.prepare('INSERT INTO users (username, email, password) VALUES (?, ?, ?)')
  .run(username, validator.normalizeEmail(email), hash);

```

Login uses a near-constant-time compare even when the user is missing, so timing does not leak which field is wrong:

```

const dummyHash = '$2b$12$abcdefghijklmnopqrstuvwxyz.invalidplaceholderhash';
const hashToCheck = user ? user.password : dummyHash;
const ok = await bcrypt.compare(password, hashToCheck);

```

2.4 Fix V-04 and V-05 — helmet() and disable powered-by**Proof**

`curl -I http://localhost:3000/` against the secure app:

```

HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';script-src 'self'; ...
Cross-Origin-Opener-Policy: same-origin
Strict-Transport-Security: max-age=15552000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: no-referrer
(no X-Powered-By)

```

2.5 Fix V-06 — JWT and hardened cookie flags

```

const token = jwt.sign({ id: user.id, username: user.username }, JWT_SECRET,
  { expiresIn: '1h' });
res.cookie('token', token, {
  httpOnly: true,
  sameSite: 'strict',
  secure: IS_PROD,
  maxAge: 60 * 60 * 1000,
});

```

Proof

Observed Set-Cookie:

```
Set-Cookie: token=eyJhbGciOi...; Max-Age=3600; Path=/; HttpOnly; SameSite=Strict
```

2.6 Fix V-07 — /users now auth-gated

```
app.get('/users', requireLogin, (req, res) => {
  const rows = db.prepare('SELECT username FROM users ORDER BY username').all();
  res.json(rows);
});
```

Anonymous requests now return 302 -> /login instead of dumping every user's email.

2.7 Fix V-08 — validator on signup

```
if (!validator.isAlphanumeric(username) || username.length < 3 || username.length > 32)
  errors.push('Username must be 3-32 alphanumeric characters.');
```

```
if (!validator.isEmail(email))
  errors.push('Invalid email address.');
```

```
if (!validator.isStrongPassword(password, {
  minLength: 8, minLowercase: 1, minUppercase: 1,
  minNumbers: 1, minSymbols: 1,
}))
  errors.push('Password must be 8+ chars with upper, lower, number, and symbol.');
```

A weak password (123) is reproducibly rejected with: *"Password must be 8+ chars with upper, lower, number, and symbol."*

2.8 Fix V-09 — Rate limit on auth endpoints

```
const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 20, ... });
app.post('/login', authLimiter, ...);
app.post('/signup', authLimiter, ...);
```

Responses include rate-limit headers:

```
RateLimit-Policy: 20;w=900
RateLimit-Limit: 20
RateLimit-Remaining: 16
```

2.9 Week 3 — Winston structured logging

logger.js creates a winston logger with two transports: a pretty console transport and a JSON-line file transport (security.log). Sample lines after one test run:

```
{"level":"info","message":"Application started","port":3000,...}
{"level":"warn","message":"Failed login",
  "username":"admin' OR '1'='1","ip":"::1",...}
{"level":"info","message":"User logged in","userId":1,
  "username":"admin",...}
{"level":"info","message":"Profile updated","userId":1,...}
```

Audit trail. The SQLi attempt is preserved verbatim in security.log, so a blue-team or SOC analyst can identify the pattern after the fact.